データベーステーブル一覧

概要

このドキュメントは、mmdpfプロジェクトで使用する全テーブルの物理名、論理名、主要カラム、DDLファイルの対応表です。

重要: DDLファイルを変更した際は、必ずこのファイルも更新してください。

最終更新: 2026-03-18 **ER図ファイル**: `docs/design/db/er_diagram.a5er` **DDLディレクトリ**: `docs/design/db/ddl/`

📋 テーブル一覧（カテゴリ別）

1. 認証関連テーブル（9テーブル）

1.1 corporation（会社）

- **物理名**: `corporation`
- **論理名**: 会社
- **DDLファイル**: `docs/design/db/ddl/corporation.sql`
- **主キー**: `corporation_id` (UUID)
- **主要カラム**:
  - `corporation_id` - 会社ID
  - `corporation_name` - 会社名
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID

- `update_at` - 更新日時

1.2 organization（組織）

- **物理名**: `organization`
- **論理名**: 組織
- **DDLファイル**: `docs/design/db/ddl/organization.sql`
- **主キー**: `organization_id` (UUID)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `organization_cd` - 組織CD
  - `organization_display_name` - 組織表示名
  - `organization_email` - 組織E-mail
  - `chief_user_id` - 代表者ユーザID
  - `corporation_id` - 会社ID (FK: corporation)
  - `is_entry` - 二段階認証判定
  - `use_deemed_progress` - みなし進捗使用フラグ
  - `contract_capacity` - 契約容量
  - `contract_users` - 契約ユーザー数
  - `contract_projects` - 契約プロジェクト数
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時
- **外部キー**:
  - `corporation_id` → `corporation(corporation_id)`

1.3 organization_user（組織利用者）

- **物理名**: `organization_user`
- **論理名**: 組織利用者
- **DDLファイル**: `docs/design/db/ddl/organization_user.sql`
- **主キー**: `user_id` (VARCHAR(50)), `organization_id` (UUID)
- **主要カラム**:
  - `user_id` - ユーザID
  - `organization_id` - 組織ID (FK: organization)
  - `user_name` - ユーザ名
  - `role_cd` - ロールCD
  - `mail_address` - メールアドレス
  - `department_cd` - 所属部署CD
  - `locked_until` - 有効期限
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時
- **外部キー**:
  - `organization_id` → `organization(organization_id)`

1.4 user_authentication（利用者認証）

- **物理名**: `user_authentication`
- **論理名**: 利用者認証
- **DDLファイル**: `docs/design/db/ddl/user_authentication.sql`

- **主キー**: `user_id` (VARCHAR(50)), `organization_id` (UUID)
- **主要カラム**:
  - `user_id` - ユーザID (FK: organization_user)
  - `organization_id` - 組織ID (FK: organization_user)
  - `password` - ハッシュパスワード
  - `entry_flg` - 二段階認証アプリ連携フラグ
  - `is_temporary_flg` - 仮パスワードフラグ
  - `password_expires_at` - パスワード有効期限
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時
- **外部キー**:
  - `(user_id, organization_id)` → `organization_user(user_id, organization_id)`

1.5 pin_codes（PINコード）

- **物理名**: `pin_codes`
- **論理名**: PINコード
- **DDLファイル**: `docs/design/db/ddl/pin_codes.sql`
- **説明**: メールPINコード二段階認証用の一時データ
- **主キー**: `organization_id` (UUID), `user_id` (VARCHAR(50))
- **主要カラム**:
  - `organization_id` - 組織ID
  - `user_id` - ユーザID
  - `pin_code` - PINコード（6桁数字、000000-999999）
  - `session_token` - セッショントークン（temp_session-{id}_{timestamp}）

  - `expires_at` - 有効期限（作成時刻 + 5分）
  - `attempts_count` - 試行回数（0-3、デフォルト: 0）
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

- **インデックス**:
  - `idx_pin_codes_token` - session_token

- **外部キー**:
  - `(user_id, organization_id)` → `organization_user(user_id, organization_id)`

1.6 login_history（ログイン履歴）

- **物理名**: `login_history`
- **論理名**: ログイン履歴
- **DDLファイル**: `docs/design/db/ddl/login_history.sql`
- **主キー**: `id` (SERIAL)
- **主要カラム**:
  - `id` - ID
  - `user_name` - ユーザ名
  - `organization_id` - 組織ID
  - `login_at` - ログイン日時
  - `login_sf` - ログイン成否
  - `failure_reason` - 失敗原因
  - `entry_execute_flg` - 二段階認証実行フラグ
  - `ip_address` - IPアドレス（セキュリティ監査用、IPv4/IPv6対応）

  - `user_agent` - User-Agent（ブラウザ・OS情報）
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

- **インデックス**:
  - `idx_login_history_rate_limit` - login_at, login_sf
  - `idx_login_history_query` - organization_id, user_name, entry_execute_flg

1.7 role（ロール）

- **物理名**: `role`
- **論理名**: ロール
- **DDLファイル**: `docs/design/db/ddl/role.sql`
- **主キー**: `organization_id` (UUID), `role_cd` (VARCHAR(30))
- **主要カラム**:
  - `organization_id` - 組織ID
  - `role_cd` - ロールCD（ユーザのロールCDと同一）
  - `role_name` - ロール名
  - `initial_screen_function_cd` - 初期画面機能CD
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

1.8 role_permission（ロール権限）

- **物理名**: `role_permission`
- **論理名**: ロール権限
- **DDLファイル**: `docs/design/db/ddl/role_permission.sql`
- **主キー**: `organization_id` (UUID), `role_cd` (VARCHAR(30)), `function_cd` (VARCHAR(30))
- **主要カラム**:
  - `organization_id` - 組織ID
  - `role_cd` - ロールCD
  - `function_cd` - 機能CD（機能名はソースで管理）
  - `permission` - 権限（0:読取専用 1:書込可）
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

1.9 department（部署）

- **物理名**: `department`
- **論理名**: 部署
- **DDLファイル**: `docs/design/db/ddl/department.sql`
- **主キー**: `organization_id` (UUID), `department_cd` (VARCHAR(50))
- **主要カラム**:
  - `organization_id` - 組織ID
  - `department_cd` - 部署CD
  - `department_name` - 部署名
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時

  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

- **外部キー**:
  - `organization_id` → `organization(organization_id)`

---

2. プロジェクト管理関連テーブル（8テーブル）

2.1 project（プロジェクト）

- **物理名**: `project`
- **論理名**: プロジェクト
- **DDLファイル**: `docs/design/db/ddl/project.sql`
- **主キー**: `organization_id` (UUID), `project_id` (CHARACTER VARYING(20))
- **主要カラム**:
  - `organization_id` - 組織ID (FK: organization)
  - `project_id` - プロジェクトID（形式: `project_` + UUID先頭12文字、例: `project_10ee60f051cb`）
  - `project_name` - プロジェクト名（プロジェクト名かつフォルダ名）
  - `description` - 概要
  - `department_cd` - 部署CD
  - `owner_user_id` - 所有者ユーザID
  - `project_status` - プロジェクトステータス
  - `project_maiml_flg` - プロジェクトMaiml有無（false:ファイルなし、true:ファイルあり）
  - `checkout_status` - チェックアウトステータス
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID

  - `update_at` - 更新日時

- **外部キー**:
  - `organization_id` → `organization(organization_id)`

2.2 project_task（プロジェクトタスク）

- **物理名**: `project_task`
- **論理名**: プロジェクトタスク
- **DDLファイル**: `docs/design/db/ddl/project_task.sql`
- **主キー**: `organization_id` (UUID), `project_id` (CHARACTER VARYING(20)), `method_id` (CHARACTER VARYING), `experiment_order` (NUMERIC(2,0))
- **主要カラム**:
  - `organization_id` - 組織ID (FK: project)
  - `project_id` - プロジェクトID (FK: project)
  - `method_id` - メソッドID
  - `experiment_order` - 実験順序
  - `experiment_condition` - 実験条件
  - `attempts_count` - 試行回数
  - `worker_linked_flg` - 作業者連携済フラグ
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

- **外部キー**:
  - `(organization_id, project_id)` → `project(organization_id, project_id)`

2.3 project_task_detail（プロジェクトタスク_詳細）

- **物理名**: `project_task_detail`
- **論理名**: プロジェクトタスク_詳細
- **DDLファイル**: `docs/design/db/ddl/project_task_detail.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `method_id` (VARCHAR), `task_no` (NUMERIC)
- **主要カラム**:
  - `organization_id` - 組織ID (FK: project_task)
  - `project_id` - プロジェクトID (FK: project_task)
  - `method_id` - メソッドID (FK: project_task)
  - `task_no` - タスク連番
  - `task_status` - タスクステータス
  - `experiment_start_at` - 実験開始日
  - `experiment_end_at` - 実験終了日
  - `experiment_user_id` - 実験作業者ID
  - `reviewed_at` - レビュー完了日
  - `review_user_id` - レビュアーID
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時
- **外部キー**:
  - `(organization_id, project_id, method_id)` → `project_task(organization_id, project_id, method_id)`

2.4 project_instruction（プロジェクト指示）

- **物理名**: `project_instruction`
- **論理名**: プロジェクト指示
- **DDLファイル**: `docs/design/db/ddl/project_instruction.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `experiment_instruction_id` (VARCHAR)
- **主要カラム**:
  - `organization_id` - 組織ID (FK: project)
  - `project_id` - プロジェクトID (FK: project)
  - `experiment_instruction_id` - 実験指示ID
  - `method_id` - メソッドID
  - `instruction_target` - 指示対象（1:ファイル全体、2:タスク、3:トランジション）
  - `priority` - 優先度
  - `type` - タイプ
  - `task_no` - タスク連番
  - `transition_id` - トランジションID
  - `instructions` - 指示内容
  - `instruction_user_id` - 実験指示ユーザID
  - `instruction_date` - 指示送信日
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時
- **外部キー**:
  - `(organization_id, project_id)` → `project(organization_id, project_id)`

2.5 instruction_acknowledged_history（指示確認履歴）

- **物理名**: `instruction_acknowledged_history`
- **論理名**: 指示確認履歴
- **DDLファイル**: `docs/design/db/ddl/instruction_acknowledged_history.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `experiment_instruction_id` (VARCHAR), `experiment_user_id` (UUID)
- **主要カラム**:
  - `organization_id` - 組織ID (PK)
  - `project_id` - プロジェクトID (PK)
  - `experiment_instruction_id` - 実験指示ID (PK)
  - `experiment_user_id` - 実験作業者ID (PK)
  - `acknowledged_at` - 確認日時
  - `create_user_id` - 作成者ユーザーID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザーID
  - `update_at` - 更新日時
- **外部キー**:
  - `(organization_id, project_id, experiment_instruction_id)` → `project_instruction(organization_id, project_id, experiment_instruction_id)`

2.6 template_data（テンプレート実績）

- **物理名**: `template_data`
- **論理名**: テンプレート実績
- **DDLファイル**: `docs/design/db/ddl/template_data.sql`

- **主キー**: `organization_id` (UUID), `project_id` (UUID), `method_id` (VARCHAR), `task_no` (NUMERIC), `place_id` (VARCHAR)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `project_id` - プロジェクトID
  - `method_id` - メソッドID
  - `task_no` - タスク連番
  - `place_id` - プレースID
  - `property_set` - プロパティセット（JSONB形式）
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

2.7 transition_data（トランジション実績）

- **物理名**: `transition_data`
- **論理名**: トランジション実績
- **DDLファイル**: `docs/design/db/ddl/transition_data.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `method_id` (VARCHAR), `task_no` (NUMERIC), `transition_id` (VARCHAR)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `project_id` - プロジェクトID
  - `method_id` - メソッドID
  - `task_no` - タスク連番
  - `transition_id` - トランジションID
  - `status` - ステータス

- `progress_rate` - 進捗率
- `create_user_id` - 作成者ユーザID
- `create_at` - 作成日時
- `update_user_id` - 更新者ユーザID
- `update_at` - 更新日時

---

3. 問題管理関連テーブル（2テーブル）

3.1 issue（問題）

- **物理名**: `issue`
- **論理名**: 問題
- **DDLファイル**: `docs/design/db/ddl/issue.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `issue_id` (VARCHAR)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `project_id` - プロジェクトID
  - `issue_id` - 問題ID
  - `issue_target` - 問題対象
  - `task_id` - タスクID
  - `transition_id` - トランジションID
  - `issue_title` - 問題タイトル
  - `issue_type` - 問題タイプ
  - `issue_contents` - 問題内容
  - `status` - ステータス
  - `issue_reporter_id` - 問題報告者ID
  - `issue_report_at` - 問題報告日時
  - `issue_completer_id` - 問題完了者ID

- `issue_complate_at` - 問題完了日時
- `create_user_id` - 作成者ユーザID
- `create_at` - 作成日時
- `update_user_id` - 更新者ユーザID
- `update_at` - 更新日時

3.2 issue_comment（問題コメント）

- **物理名**: `issue_comment`
- **論理名**: 問題コメント
- **DDLファイル**: `docs/design/db/ddl/issue_comment.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `issue_id` (VARCHAR), `sequence_no` (NUMERIC)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `project_id` - プロジェクトID
  - `issue_id` - 問題ID
  - `sequence_no` - シーケンス番号
  - `comment` - コメント
  - `author_id` - 記入者ID
  - `commented_at` - 記入日時
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

---

4. 共通マスタテーブル（5テーブル）

4.1 code_master（コードマスタ）

- **物理名**: `code_master`
- **論理名**: コードマスタ
- **DDLファイル**: `docs/design/db/ddl/code_master.sql`
- **主キー**: `organization_id` (UUID), `code_division` (VARCHAR(30)), `code_value` (VARCHAR(30))
- **主要カラム**:
  - `organization_id` - 組織ID
  - `code_division` - コード区分
  - `code_value` - コード値
  - `code_description` - コード概要
  - `code_name` - コード名称
  - `code_remark` - 備考
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

4.2 creator（creator）

- **物理名**: `creator`
- **論理名**: creator
- **DDLファイル**: `docs/design/db/ddl/creator_master.sql`
- **主キー**: `organization_id` (UUID), `creator_uuid` (UUID)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `creator_uuid` - creator_uuid
  - `vendor_uuid` - vendor_uuid

- `instrument_uuid` - instrument_uuid
- `corporation_id` - 会社ID
- `creator_id` - ID
- `creator_name` - 名称
- `creator_description` - 説明
- `creator_annotation` - 注釈
- `is_active` - アクティブ判定
- `create_user_id` - 作成者ユーザID
- `create_at` - 作成日時
- `update_user_id` - 更新者ユーザID
- `update_at` - 更新日時

4.3 instrument（instrument）

- **物理名**: `instrument`
- **論理名**: instrument
- **DDLファイル**: `docs/design/db/ddl/instrument_master.sql`
- **主キー**: `organization_id` (UUID), `instrument_uuid` (UUID)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `instrument_uuid` - uuid
  - `corporation_id` - 会社ID
  - `instrument_id` - ID
  - `instrument_name` - 名称
  - `instrument_description` - 説明
  - `instrument_annotation` - 注釈
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID

- `create_at` - 作成日時
- `update_user_id` - 更新者ユーザID
- `update_at` - 更新日時

4.4 vendor（vendor）

- **物理名**: `vendor`
- **論理名**: vendor
- **DDLファイル**: `docs/design/db/ddl/vendor_master.sql`
- **主キー**: `organization_id` (UUID), `vendor_uuid` (UUID)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `vendor_uuid` - uuid
  - `corporation_id` - 会社ID
  - `vendor_id` - ID
  - `vendor_name` - 名称
  - `vendor_description` - 説明
  - `vendor_annotation` - 注釈
  - `is_active` - アクティブ判定
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID
  - `update_at` - 更新日時

4.5 attribute（属性）

- **物理名**: `attribute`
- **論理名**: 属性

- **DDLファイル**: `docs/design/db/ddl/attribute.sql`
- **主キー**: `organization_id` (UUID), `attribute_id` (VARCHAR)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `attribute_id` - 属性ID
  - `attribute_name` - 属性名
  - `attribute_value` - 属性値
  - `create_user_id` - 作成者ユーザID
  - `update_user_id` - 更新者ユーザID
  - `create_at` - 作成日時
  - `update_at` - 更新日時

---

5. 監査・ログ関連テーブル（1テーブル）

5.1 maiml_event_log（MaiMLイベントログ）

- **物理名**: `maiml_event_log`
- **論理名**: MaiMLイベントログ
- **DDLファイル**: `docs/design/db/ddl/maiml_event_log.sql`
- **主キー**: `organization_id` (UUID), `project_id` (UUID), `sequence_no` (NUMERIC)
- **主要カラム**:
  - `organization_id` - 組織ID
  - `project_id` - プロジェクトID
  - `sequence_no` - シーケンス番号
  - `log` - ログ（JSONB形式）
  - `create_user_id` - 作成者ユーザID
  - `create_at` - 作成日時
  - `update_user_id` - 更新者ユーザID

- `update_at` - 更新日時

---

6. ファイル管理関連テーブル（1テーブル）

6.1 file_metadata（ファイルメタデータ）

- **物理名**: `file_metadata`
- **論理名**: ファイルメタデータ
- **DDLファイル**: `docs/design/db/ddl/file_metadata.sql`
- **説明**: S3/Swiftファイルのメタデータ管理テーブル（v1.0 - 初期実装: S3アクセス必須カラムのみ）
- **主キー**: `file_key` (VARCHAR(500)) - S3キーを主キーとする自然キー設計
- **主要カラム**:
  - `file_key` - S3キー（org_prefix/path/to/file形式、主キー）
  - `s3_bucket` - S3バケット名またはSwiftコンテナ名
  - `organization_id` - 組織ID（マルチテナント分離用、必須）
  - `original_filename` - 元のファイル名（アップロード時の名前）
  - `file_size` - ファイルサイズ（バイト単位）
  - `file_type` - MIMEタイプ（例: application/xml）
  - `uploaded_by` - アップロードユーザーID
  - `created_at` - 作成日時（自動設定）
  - `updated_at` - 更新日時（トリガーで自動更新）
- **外部キー**: なし
- **インデックス**:
  - `idx_file_metadata_organization_id` - organization_id（マルチテナント分離必須）
  - `idx_file_metadata_uploaded_by` - uploaded_by（ユーザー単位のファイル一覧取得用）

- **トリガー**:
  - `update_file_metadata_updated_at` - updated_at自動更新
- **特記事項**:
  - file_keyは主キーなので、インデックスは自動作成される
  - すべてのクエリでorganization_idフィルタを必須とする
  - v2.0でMAIML解析メタデータ追加予定

---

7. 公開ジョブ管理テーブル（1テーブル、publicスキーマ）

7.1 maiml_publish_job（MaiML公開ジョブ）

- **物理名**: `maiml_publish_job`
- **論理名**: MaiML公開ジョブ
- **スキーマ**: `public`（全組織共通）
- **DDLファイル**: `docs/design/db/ddl/maiml_publish_job.sql`
- **マイグレーション**: `backend/migrations/20260311_create_maiml_publish_job.sql`
- **主キー**: `job_id` (UUID)
- **主要カラム**:
  - `job_id` - ジョブID（UUID、BullMQペイロードにも含める）
  - `job_type` - ジョブ種別（`publish` / `unpublish`）
  - `organization_id` - 組織ID（マルチテナント分離用）
  - `project_name` - プロジェクト名
  - `asset_id` - maiml_assetのasset_id（FK制約なし、クロススキーマのため）
  - `status` - ステータス（`pending` / `running` / `done` / `error`）
  - `triggered_by` - 実行者ユーザーID（スケジュール実行時はNULL）
  - `trigger_type` - 実行種別（`manual` / `schedule`）
  - `error_message` - エラー内容（status='error'時）

  - `started_at` - 処理開始日時
  - `finished_at` - 処理完了日時
  - `created_at` - レコード作成日時

- **インデックス**:
  - `ix_publish_job_org_project` - organization_id, project_name
  - `ix_publish_job_status` - status
  - `ix_publish_job_type_status` - job_type, status

- **ステータス遷移**: `pending` → `running` → `done` / `error`
- **備考**: asset_idはクロススキーマのためFK制約なし。参照整合性はアプリケーション側で管理

---

8. 公開データテーブル（12テーブル、組織スキーマ）

> ***重要**: 以下のテーブルは `public` スキーマではなく、**組織ごとに作成される専用スキーマ**に配置される。*
>
> - *スキーマ命名規則: `organization_id` の `-` を `_` に変換した文字列（例: `550e8400_e29b_41d4_a716_446655440000`）*
> - *スキーマ作成タイミング: 組織作成時（`POST /api/v1/provider/organization`）*
> - *マイグレーション: `backend/migrations/20260313_create_org_schema_tables.sql`（data/protocol 層）、`backend/migrations/20260318_add_event_tables_to_org_schema.sql`（eventLog 層）*
> - *DDL詳細: `docs/MaiML_OpenData_Pipeline/db.md` を参照*

8.1 maiml_asset（MaiMLアセット）

- **物理名**: `maiml_asset`
- **スキーマ**: 組織スキーマ（例: `550e8400_e29b_41d4_a716_446655440000`）
- **主キー**: `asset_id` (BIGSERIAL)

- **役割**: MaiMLファイル1件の代表メタ（1ファイル = 1レコード）
- **主要カラム**: `file_uri`, `doc_name`, `retention_until`, `created_at`, `updated_at`

8.2 maiml_instance（MaiMLインスタンス）

- **物理名**: `maiml_instance`
- **主キー**: `instance_id` (BIGSERIAL)
- **役割**: result/material/condition インスタンス（data層）
- **主要カラム**: `asset_id` (FK), `instance_type` (`result` / `material` / `condition`), `instance_key`, `results_id`, `template_ref`

8.3 maiml_element（MaiML要素）

- **物理名**: `maiml_element`
- **主キー**: `element_id` (BIGSERIAL)
- **役割**: 汎用データ行（property/content/uncertainty/insertion系）
- **主要カラム**: `instance_id` (FK), `parent_element_id` (自己参照FK), `hierarchy`, `element_type`, `key`, `units`

8.4 maiml_element_value（MaiML要素値）

- **物理名**: `maiml_element_value`
- **主キー**: `value_id` (BIGSERIAL)
- **役割**: `<value>` 子要素の展開（複数value対応）
- **主要カラム**: `element_id` (FK), `value_idx`, `value` (単一値), `value_array` (配列値)

8.5 maiml_insertion（MaiML外部参照）

- **物理名**: `maiml_insertion`
- **主キー**: `insertion_id` (BIGSERIAL)

- **役割**: 外部ファイル参照（正規化）
- **主要カラム**: `instance_id` (FK), `uri`, `hash_value`, `mime_format`

8.6 maiml_template（MaiMLテンプレート）

- **物理名**: `maiml_template`
- **主キー**: `template_id` (BIGSERIAL)
- **役割**: テンプレート定義（protocol層）
- **主要カラム**: `asset_id` (FK), `template_type` (`result` / `material` / `condition`), `template_key`

8.7 maiml_template_element（MaiMLテンプレート要素）

- **物理名**: `maiml_template_element`
- **主キー**: `element_id` (BIGSERIAL)
- **役割**: テンプレートの汎用データ行
- **主要カラム**: `template_id` (FK), `parent_element_id` (自己参照FK), `hierarchy`, `element_type`, `key`

8.8 maiml_template_element_value（MaiMLテンプレート要素値）

- **物理名**: `maiml_template_element_value`
- **主キー**: `value_id` (BIGSERIAL)
- **役割**: テンプレートの `<value>` 展開
- **主要カラム**: `element_id` (FK), `value_idx`, `value`, `value_array`

8.9 maiml_template_insertion（MaiMLテンプレート外部参照）

- **物理名**: `maiml_template_insertion`
- **主キー**: `insertion_id` (BIGSERIAL)
- **役割**: テンプレートの外部ファイル参照（正規化）
- **主要カラム**: `template_id` (FK), `uri`, `hash_value`, `mime_format`

8.10 maiml_event（MaiMLイベント）

- **物理名**: `maiml_event`
- **主キー**: `event_id` (BIGSERIAL)
- **役割**: event要素（eventLog層）。 `results_ref` でdata層と論理結合
- **主要カラム**: `asset_id` (FK), `event_key` , `results_ref` （maiml_instance.results_idと結合）, `instruction_ref`

8.11 maiml_event_element（MaiMLイベント要素）

- **物理名**: `maiml_event_element`
- **主キー**: `event_element_id` (BIGSERIAL)
- **役割**: event配下のproperty / content行（maiml_elementと同一カラム構成）
- **主要カラム**: `event_id` (FK), `hierarchy` , `element_type` (`property` / `content`), `description` , `xsi_type` , `key` , `format_string` , `units` , `scale_factor` , `axis` (contentのみ), `size_value` (contentのみ), `element_id_attr` (contentのみ), `element_ref` (contentのみ)
- **マイグレーション**: `backend/migrations/20260319_add_columns_to_event_element.sql` （2026-03-19 カラム追加）

8.12 maiml_event_element_value（MaiMLイベント要素値）

- **物理名**: `maiml_event_element_value`
- **主キー**: `value_id` (BIGSERIAL)
- **役割**: propertyの `<value>` 展開
- **主要カラム**: `event_element_id` (FK), `value_idx` , `value` , `value_array`

📊 テーブル数サマリー

カテゴリ	テーブル数
認証関連	8
プロジェクト管理関連	8
問題管理関連	2
共通マスタ	5
監査・ログ関連	1
ファイル管理関連	1
公開ジョブ管理（publicスキーマ）	1
公開データ（組織スキーマ）	12
合計（publicスキーマ）	**26**
合計（組織スキーマ）	**12**

🔗 主要な外部キー関係

認証フロー

```
corporation
  ↓
organization
  ↓
organization_user
  ↓ ↓
  | └→ user_authentication
  └→ pin_codes

role_permission
```

プロジェクト管理フロー

```
organization
  ↓
project
  ↓ ↓
  | └→ project_instruction
```

```
  │      ↓
  │      instruction_acknowledged_history
  ↓
project_task
  ↓ ↓
  │ └↪ template_data
  └↪ transition_data
  ↓
project_task_detail

project
  ↓
issue
  ↓
issue_comment
```

ログフロー

```
project
  ↓
maiml_event_log
```

📌 重要な設計原則

1. マルチテナント分離

- **すべてのテーブル**に `organization_id` カラムを含める（corporation以外）
- すべてのクエリに `organization_id` フィルタを必須とする
- クロステナントアクセスを防ぐ

2. UUID使用

- 主要な識別子（organization_id, user_id, project_id等）にUUIDを使用
- セキュリティと一意性を確保

3. タイムスタンプ

- 作成日時: `create_at` / `created_at`
- 更新日時: `update_at` / `updated_at`
- テーブルによって命名が異なるため注意

4. インデックス設計

- 外部キーカラムに必ずインデックスを作成
- 検索頻度の高いカラムにインデックスを作成
- JSONB型カラムにはGINインデックスを使用

📝 更新履歴

日付	変更内容	担当者
2026-02-27	project.project_id型をUUIDからCHARACTER VARYING(20)に変更（MAIML DocumentData.idとの連携対応）	Cline
	project_id形式: `project_` + UUID先頭12文字（例: `project_10ee60f051cb`）、最大20文字	
	マイグレーション: `backend/migrations/20260227_alter_project_id_to_varchar.sql`	
2026-02-10	login_history.login_sfカラムをCHARACTER VARYING(10)からBOOLEANに変更（型整合性確保）	Cline
2026-02-10	login_historyテーブルにentry_execute_flg、idx_login_history_queryを追加（ER図更新に伴う）	Cline
2026-02-09	ER図更新に合わせて全テーブルの監査カラム（create_user_id, update_user_id等）を追加	Cline
2026-01-31	instruction_acknowledged_historyテーブル詳細情報を更新（ER図に基づく）	Cline
2026-01-29 17:46	function, user_notificationテーブル削除（25テーブルに統一）	Cline
2026-01-29 17:31	ER図の最新構造に合わせて更新（role, attribute, instrument, vendor）	Cline
2026-01-29 16:40	DDLファイル（27個）の現状に合わせて更新完了	Cline
2026-01-23 15:00	event_log.sql削除に伴い27テーブルに更新	Cline

2026-01-23 14:59	DDLファイル(28個)と完全一致するように整理、重複削除、カテゴリ整理	Cline
2026-01-21	DDLファイル更新に伴い29テーブルに更新	Cline
2026-01-16	file_metadataテーブル追加（ファイル管理関連カテゴリ新設）	Cline
2026-01-09	初版作成（24テーブル）	Cline

✅ DDL配下ファイルとの整合性

DDL配下のファイル（25個）：

1. attribute.sql
2. code_master.sql
3. corporation.sql
4. creator_master.sql
5. department.sql
6. file_metadata.sql
7. instruction_acknowledged_history.sql
8. instrument_master.sql
9. issue_comment.sql
10. issue.sql
11. login_history.sql
12. maiml_event_log.sql
13. organization_user.sql
14. organization.sql
15. pin_codes.sql
16. project_instruction.sql
17. project_task_detail.sql
18. project_task.sql
19. project.sql
20. role_permission.sql
21. role.sql
22. template_data.sql
23. transition_data.sql

24. user_authentication.sql
25. vendor_master.sql

このドキュメントのテーブル数: 25個 ✅

すべてのDDLファイルがこのドキュメントに反映されています。